DATABASE SYSTEMS · EXPERIMENT 5

Large-scale Document Retrieval System

Implementation in MongoDB with GridFS

Experiment report

Student name: _____

Student ID: _____

23 October 2026

Source code: https://github.com/wingstech2026-byte/document-retrieval

Contents

Abstract 3
1 Introduction 3
1.1 Background and task 3
1.2 Objectives 3
1.3 What was built 3
1.4 Structure of this report 4
2 Requirements analysis 4
2.1 Document types 4
2.2 Functional requirements 4
2.3 Non-functional requirements 5
3 Technology choices 5
3.1 MongoDB 5
3.2 GridFS 5
3.3 Python, pymongo and Flask 6
3.4 Cloud deployment 6
3.5 Alternatives considered 6
4 System design 6
4.1 Architecture 6
4.2 Data model 7
4.3 Collections 8
4.4 Indexes 10
4.5 Design decisions 10
4.6 Consistency without transactions 10
5 Implementation 11
5.1 Code organisation 11
5.2 Text analysis 11
5.3 Document storage 12
5.4 Document retrieval 13
5.5 Document reading 16
5.6 Document comparison 17
5.7 Association analysis 18
5.8 Document download 19
5.9 Deletion 20

5.10 Statistics 20
5.11 Command-line interface 21
5.12 Web interface and JSON API 21
5.13 Additional features 22
5.14 Cloud deployment 24
6 Testing and evaluation 24
6.1 Test strategy 24
6.2 Checking that the tests can fail 25
6.3 Demonstration on the sample data 26
6.4 Performance at scale 26
7 Discussion 29
7.1 What worked well 29
7.2 Limitations 29
7.3 Future work 29
8 Conclusion 30
References 30
Appendix A: Command reference 30
Appendix B: Sample documents 31
Appendix C: Benchmark tables 32
Appendix D: Running the system 32

Abstract

This report describes the design, implementation and evaluation of a large-scale document retrieval system built on MongoDB. The system stores four kinds of documents (papers, patents, technical reports and engineering drawings) and provides document storage, retrieval, reading, comparison, association analysis and download.

Metadata and text are kept in MongoDB collections, the original files, including binary drawings, in GridFS, and a custom inverted index in a `postings` collection with a unique index on (term, document). Ranked search uses BM25 with filters and snippets. Comparison uses TF-IDF cosine similarity. Association analysis provides related documents, a similarity graph and keyword co-occurrence counts computed by a MongoDB aggregation pipeline.

The system can be used from a command line, a web site and a JSON API, and is deployed in the cloud. 127 automated tests pass against a real MongoDB server. A benchmark with up to 10,000 documents (986,895 index entries) shows a constant ingest rate of about 194 documents per second, search times from 19 ms to 181 ms depending on how common the query words are, and an index lookup of 4 ms against 479 ms for a scan. The measurements also reveal limits, which are discussed together with proposed improvements.

Keywords: MongoDB, GridFS, inverted index, BM25, TF-IDF, document retrieval, NoSQL.

1 Introduction

1.1 Background and task

The task for Experiment 5 of the course is to implement a “large-scale document retrieval system” in a NoSQL product. The system collects and stores paper documents, patent documents, technical report documents and engineering drawing documents, and offers document retrieval, reading, comparison, association analysis, download and storage.

1.2 Objectives

- Use at least one NoSQL product: here MongoDB, with the `pymongo` driver and GridFS.
- Handle four document types, with drawings stored as binary files.
- Provide the six required functions, plus deletion and statistics.
- Keep the original files and a searchable index consistent with each other, including when documents are deleted.
- Show by measurement that the design works on a collection of thousands of documents.

1.3 What was built

- A document store: metadata and text in `documents`, original files in GridFS, an inverted index in `postings` (one record per term and document, unique index on the pair).
- Ranked search with BM25, filters by type, year range and author, text snippets, a query language, facets and spelling suggestions.

- Paginated reading, TF-IDF cosine comparison, related documents, a similarity graph, keyword co-occurrence and download with checksum verification.
- Deletion that removes the index entries and the stored file, and a statistics report.
- Three interfaces over one engine: a command-line tool (12 required commands plus 5 more), a Flask web site and a JSON API, deployed with Vercel and MongoDB Atlas.
- 127 automated tests and a scale benchmark.

The source code is available at https://github.com/wingstech2026-byte/document-retrieval.

1.4 Structure of this report

Section 2 analyses the requirements and Section 3 explains the technology choices. Section 4 presents the design: architecture, data model, collections and indexes. Section 5 describes the implementation of each function. Section 6 reports the tests and the scale measurements. Section 7 discusses limits and future work, and Section 8 concludes. The appendices give the command reference, the sample data, the full benchmark tables and instructions for running the system.

2 Requirements analysis

2.1 Document types

Table 1. *The four document types and how each is stored and searched.*

Type	Typical content	Stored as	Searchable through
Paper	title, authors, abstract, body text	text in `documents` plus the original file in GridFS	title, keywords, abstract, body
Patent	title, inventors, claims, description	text in `documents` plus the original file	title, keywords, abstract, claims and description
Technical report	numbered sections, results, recommendations	text in `documents` plus the original file	title, keywords, abstract, body
Engineering drawing	a binary file (PNG, DXF, PDF) and a short description	description in `documents`, the binary file in GridFS	title, keywords and description

2.2 Functional requirements

Table 2. *Functional requirements and where they are implemented.*

ID	Requirement	Implementation
F1	Document storage: store metadata, text and the original file	`add` command, `DocumentStore.add_document`
F2	Document retrieval: ranked search with filters by type, year range and author, and a text snippet for each result	`search` command, `search_ex` (BM25)

ID	Requirement	Implementation
F3	Document reading: paginated	`read` command, `DocumentStore.read`
F4	Document comparison: similarity, shared and distinctive terms, metadata overlap	`compare` command (TF-IDF cosine)
F5	Association analysis: related documents, a similarity graph above a threshold, keyword co-occurrence	`related`, `graph`, `cooccur` commands
F6	Document download: write the original file from GridFS to a folder	`download` command
F7	Delete a document together with its index entries and its GridFS file	`delete` command
F8	Statistics about the collections, the index and the stored files	`stats` command

2.3 Non-functional requirements

- **Scalability:** lookups must use indexes, so that the cost of a search depends on the number of matching index entries and not on the size of the whole collection.
- **Integrity:** a failed add must not leave partial data; a delete must leave no orphan index entry or file; a downloaded file must be identical to the stored one.
- **Usability:** the same functions from a command line, a web browser and a program (JSON).
- **Security:** stored text is escaped in web pages, uploaded files cannot run scripts in the site, and a public deployment can be made read-only.
- **Testability:** each requirement is covered by automated tests that run against a real MongoDB server.

3 Technology choices

3.1 MongoDB

MongoDB is a document database. A document is a JSON-like record whose fields can be strings, numbers, dates, arrays and nested objects. This suits the problem: the four document types share most fields but differ in detail (a drawing has no body text, a patent has claims), and authors and keywords are lists that belong inside the record. A secondary index on any field, including a compound unique index, is available, and the aggregation framework computes counts and groupings inside the database.

3.2 GridFS

A MongoDB record is limited to 16 MB and is not intended for large binary data. GridFS is MongoDB's standard way to store files: a file is cut into chunks of 255 KB, stored in the collection `fs.chunks`, and described by one record in `fs.files`. The files therefore live in the same database as the metadata, so a backup or a deletion covers both.

3.3 Python, pymongo and Flask

The system is written in Python 3.12. `pymongo` is the official MongoDB driver and includes the GridFS API. The web interface uses Flask. The command-line tool uses the standard library module `argparse`. Tests use `pytest`.

3.4 Cloud deployment

The web site is deployed on Vercel, which builds the Flask application from the GitHub repository, and the data is held in MongoDB Atlas, the hosted version of MongoDB. The connection string is supplied as an environment variable and is never part of the source code.

3.5 Alternatives considered

Table 3. *Alternatives that were considered and not chosen.*

Alternative	Reason for not choosing it
Keep files on disk and store only their paths in the database	A delete could leave an orphan file, and the database and the files would not be backed up together. GridFS avoids both problems.
Use MongoDB's built-in `$text` index	Its scoring and options are fixed. A custom inverted index allows BM25 with field weights, phrase and exclusion handling, facets and an explanation of each score, which is also the point of the experiment.
A relational database (for example PostgreSQL)	The assignment requires a NoSQL product, and the variable structure of the documents fits a document model better.
A search server such as Elasticsearch	It would add a second system to run and keep in step with the data. The experiment is about MongoDB.

4 System design

4.1 Architecture

Figure 1 shows the architecture. There are three ways to use the system: a web browser, other programs through the JSON API, and the command line. All three call one Python class, `DocumentStore`, which is the only code that talks to MongoDB. A function therefore has to be written only once, and the tests can exercise the engine directly.

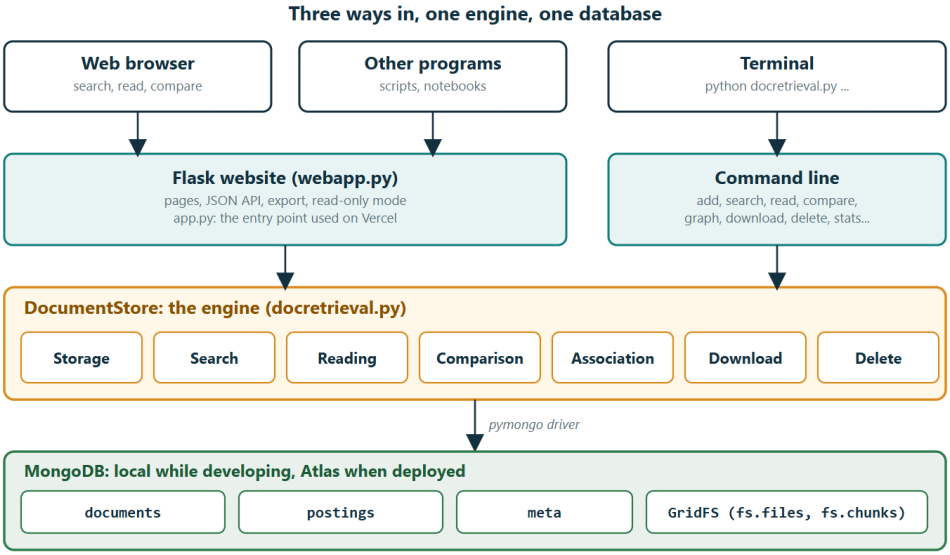


Figure 1. *Architecture: three interfaces, one engine (`DocumentStore`), one database.*

4.2 Data model

Figure 2 is the entity-relationship diagram. Solid boxes are MongoDB collections. Authors and keywords are drawn as dashed boxes because they are not separate collections: they are lists embedded in each document. Terms are also dashed: there is no collection of terms, the vocabulary is the set of distinct `term` values in `postings`.

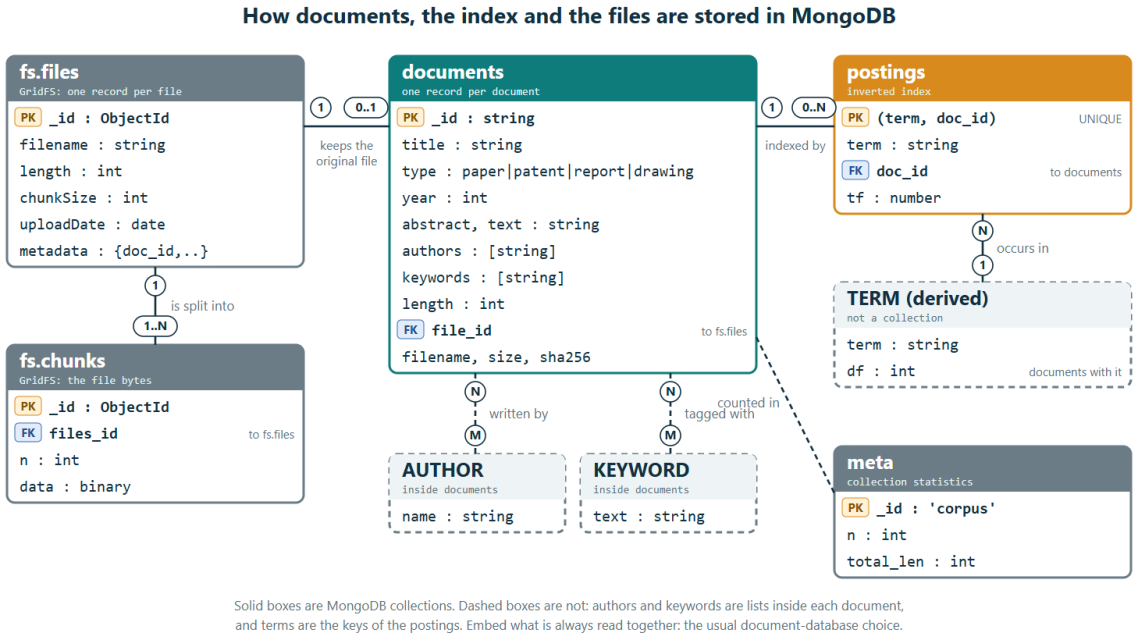


Figure 2. *Entity-relationship diagram of the collections, GridFS and the embedded or derived concepts.*

Table 4. *The relationships of the data model.*

Relationship	Cardinality	Implemented by
document – index entries (postings)	1 : 0..N	`postings.doc_id` refers to `documents._id`

Relationship	Cardinality	Implemented by
document – original file	0..1 : 1	`documents.file_id` refers to `fs.files._id`
file – chunks	1 : 1..N	`fs.chunks.files_id` refers to `fs.files._id`
document – authors	N : M	embedded list `documents.authors`
document – keywords	N : M	embedded list `documents.keywords`
term – index entries	1 : N	`postings.term` (no separate collection)

4.3 Collections

The following tables are the data dictionary. The `documents` collection holds one record per document of any type.

Table 5. *Fields of the `documents` collection.*

Field	Type	Description
`_id`	string	readable unique id, for example `pat-001`; the first letters show the type
`title`	string	title of the document
`type`	string	`paper`, `patent`, `report` or `drawing`
`authors`, `keywords`	array of strings	embedded lists; keywords are stored in lower case
`year`	integer	publication year (optional)
`abstract`, `text`	string	abstract or description, and the full text used for search and reading
`length`	integer	number of weighted terms, needed for BM25 length normalisation
`file_id`	ObjectId	link to the original file in GridFS (null if there is none)
`filename`, `content_type`, `size`	string, string, integer	information about the original file
`sha256`	string	fingerprint of the original file, checked on download
`added_at`	date	time of storage

Table 6. *Fields of the `postings` collection (the inverted index).*

Field	Type	Description
`term`	string	an index term (lower case, plural ending removed)
`doc_id`	string	the document that contains the term
`tf`	number	weighted count of the term in the document

The pair (`term`, `doc_id`) is the key of a posting. The collection `meta` holds a single record, `{_id: "corpus", n, total_len}`, with the number of documents and the sum of their lengths, which BM25 needs for the average document length. GridFS adds `fs.files` (`_id`, `filename`, `length`, `chunkSize`, `uploadDate`, `metadata`) and `fs.chunks` (`files_id`, `n`, `data`).

```
// db.documents.findOne({_id: "pat-001"})
{
  "_id": "pat-001",
  "title": "Method for Compressing Posting Lists Using Block-wise Delta Coding",
  "type": "patent",
  "authors": [
    "Sofia Alvarez"
  ],
  "keywords": [
    "inverted index",
    "compression",
    "posting list",
    "information retrieval",
    "delta coding"
  ],
  "year": 2020,
  "abstract": "A method of storing the document identifiers of an inverted index in fixed-size blocks, ea...",
  "text": "Claim 1. A method of storing a posting list of an inverted index, the posting list compris...",
  "length": 199,
  "file_id": "ObjectId('6adb10a93ff257772904339d')",
  "filename": "pat-001.txt",
  "content_type": "text/plain",
  "size": 1457,
  "sha256": "599ee1e827c1545f5617b6d8e8ad812e366b515e4238d9a473fe8906cbf72819",
  "added_at": "2026-10-23T07:45:45.311000"
}
```

Figure 3. *A real `documents` record (patent `pat-001`), with the text shortened.*

```
// db.postings.find({doc_id: "pat-001", term: {$in: [...]}})
{"term": "compression", "doc_id": "pat-001", "tf": 2}
{"term": "inverted", "doc_id": "pat-001", "tf": 4}
{"term": "posting", "doc_id": "pat-001", "tf": 7}

// db.meta.findOne({_id: "corpus"})
{"_id": "corpus", "n": 10, "total_len": 1617}
```

Figure 4. *Real `postings` records for `pat-001`, and the `meta` record of the demo database.*

```
// db.fs.files.findOne({filename: "gearbox_housing_sheet3.png"})
{
  "_id": "ObjectId('6adb10a93ff2577729043542')",
  "filename": "gearbox_housing_sheet3.png",
  "length": 1016,
  "chunkSize": 261120,
  "uploadDate": "2026-10-23T07:45:45.349000",
  "metadata": {
    "doc_id": "drw-001",
    "content_type": "image/png",
    "doc_type": "drawing"
  }
}

// db.fs.chunks.findOne({files_id: ...})
{
  "files_id": "ObjectId('6adb10a93ff2577729043542')",
  "n": 0,
  "data": "<Binary: 1016 bytes, PNG signature b'\\x89PNG'>"
}
```

Figure 5. *A real GridFS file record and its single chunk (a 1,016-byte PNG drawing).*

4.4 Indexes

Table 7. *Indexes created automatically when the program connects.*

Collection	Index	Purpose
`postings`	(`term`, `doc_id`), **unique**	finds all documents of a term quickly; guarantees one record per term and document
`postings`	`doc_id`	rebuilds the term vector of one document; removes all postings of a deleted document
`documents`	(`type`, `year`), `year`, `authors`, `keywords`	filters and keyword counts

```
// db.postings.getIndexes()   and   db.documents.getIndexes()
postings
  _id_                  {_id: 1}
  term_doc_unique  UNIQUE {term: 1, doc_id: 1}
  doc_id                {doc_id: 1}
documents
  _id_                  {_id: 1}
  type_year             {type: 1, year: 1}
  authors               {authors: 1}
  keywords              {keywords: 1}
  year                  {year: 1}
```

Figure 6. *The indexes reported by MongoDB for `postings` and `documents`.*

4.5 Design decisions

- **Authors and keywords are embedded.** They are always read together with the document and do not change on their own, so one read returns a complete document and no join is needed.
- **The inverted index is a separate collection.** A term can occur in thousands of documents. A list inside each term record would grow without limit, and a search needs “all documents of a term” directly from an index.
- **Weighted term counts.** A word counts three times in the title, twice in a keyword and once in the abstract or text, so that title and keyword matches rank higher.
- **Collection counters.** `meta` keeps the document count and total length, updated atomically with `$inc`, so ranking never has to scan the collection.
- **A checksum for every file.** The SHA-256 of the original file is stored, and a download is rejected if it differs.
- **Readable document ids.** For example `drw-001`, so that users and logs can tell the type at a glance.

4.6 Consistency without transactions

A single MongoDB server does not support transactions over several documents, so consistency is kept by the order of the writes. When a document is added, the file is written first, then the record, then the postings and finally the counters; if any step fails, everything already written is removed again. When a document is deleted, the postings are removed first, so the document can no longer be found, then the file, then the record and the counters. A half-finished delete never leaves a

document that can be found but not opened, and running it again completes it. A repair function can recompute the counters from the documents.

5 Implementation

5.1 Code organisation

Table 8. *Files of the project.*

File	Purpose	Lines
`docretrieval.py`	the engine (`DocumentStore`) and the command-line interface	1,412
`webapp.py`, `app.py`	the Flask web site and the entry point for Vercel	456
`templates/`	14 page templates and the style sheet	about 460
`sample_data.py`	ten original sample documents, including two generated PNG drawings	257
`test_*.py` (4 files)	automated tests	1,126

5.2 Text analysis

Before anything is indexed, the text is analysed by `tokenize`: it is converted to lower case and split at every character that is not a letter or digit; stop words and one-letter words are dropped; and plural endings are removed, so that “gearboxes” becomes “gearbox”. `index_terms` then counts the resulting terms with the weights described above. The same function analyses a search query, which is why a search for “turbine” finds a document that says “turbines”.

```
91  def tokenize(text: str) -> list[str]:
92      """Lower-case, split on non-alphanumerics, drop stop words, stem."""
93      out = []
94      for tok in _TOKEN_RE.findall((text or "").lower()):
95          if len(tok) < 2 or tok in STOPWORDS:
96              continue
97          out.append(stem(tok))
98      return out
99
100
101 def index_terms(title: str, keywords: list[str], abstract: str, text: str) -> Counter:
102     """Weighted term frequencies: title x3, keywords x2, abstract/body x1."""
103     counts: Counter = Counter()
104     for t in tokenize(title):
105         counts[t] += TITLE_BOOST
106     for kw in keywords:
107         for t in tokenize(kw):
108             counts[t] += KEYWORD_BOOST
109     for t in tokenize(abstract):
110         counts[t] += 1
111     for t in tokenize(text):
112         counts[t] += 1
113     return counts
```

Figure 7. *Text analysis: `tokenize` and `index_terms` (`docretrieval.py`, lines 91–113).*

5.3 Document storage

`add_document` stores a document in the steps of Table 9. A document can be given its text directly, or the text is read from a `.txt` or `.pdf` file; for a drawing, its description is its text. If anything is invalid (empty title, unknown type, implausible year, id already used), nothing is written.

Table 9. *Steps of `add_document`.*

Step	What happens
1 Validate	title not empty, type one of the four, year plausible, id not in use
2 Get the text	the text given; otherwise extracted from a `.txt` or `.pdf` file; otherwise the description
3 Analyse	lower-case, split, drop stop words, strip plural endings
4 Weight	title words ×3, keyword words ×2, abstract and text ×1; the document length is the sum of the counts
5 Write the file	the original file is uploaded to GridFS; its name, type, size and SHA-256 are added to the record
6 Write the record	the record is inserted into `documents`
7 Write the index	one posting per distinct term, in one `insert_many`
8 Update the counters	`meta` is updated atomically with `$inc`; any error before this point undoes all earlier writes

As an example, the engineering drawing “Gearbox Housing Assembly Drawing, Sheet 3” (author Tomas Berg, year 2023, five keywords, a 1,016-byte PNG file) produces one GridFS file, one `documents` record with length 95, and 49 postings. The term `gearbox` receives the weighted count 7: 3 for the title, 2 for the keyword and 1 each for the abstract and the text.

```
431 file_id = None
432 try:
433     if file_bytes is not None:
434         filename = _safe_filename(filename or f"{doc_id}.bin")
435         ctype = mimetypes.guess_type(filename)[0] or "application/octet-stream"
436         file_id = self.fs.upload_from_stream(
437             filename, io.BytesIO(file_bytes),
438             metadata={"doc_id": doc_id, "content_type": ctype, "doc_type": doc_type})
439         record.update(file_id=file_id, filename=filename, content_type=ctype, size=len(file_bytes),
440                       sha256=hashlib.sha256(file_bytes).hexdigest())
441     self.documents.insert_one(record)
442     if terms:
443         self.postings.insert_many(
444             [{"term": t, "doc_id": doc_id, "tf": tf} for t, tf in terms.items()], ordered=False)
445     self.meta.update_one({"_id": "corpus"}, {"$inc": {"n": 1, "total_len": record["length"]}}, upsert=True)
446 except Exception:
447     # No multi-document transactions on a standalone server: undo by hand.
448     self.postings.delete_many({"doc_id": doc_id})
449     self.documents.delete_one({"_id": doc_id})
450     if file_id is not None:
451         try:
452             self.fs.delete(file_id)
453         except gridfs.errors.NoFile:
454             pass
455     raise
456 self._vocab = None
457 return {"doc_id": doc_id, "type": doc_type, "terms": len(terms), "length": record["length"],
458         "file_id": file_id, "size": record["size"]}
```

Figure 8. *The write part of `add_document` with its rollback (`docretrieval.py`, lines 431–458).*

5.4 Document retrieval

5.4.1 Ranking with BM25

Documents are ranked with BM25 (Robertson and Zaragoza, 2009). For a query, the score of a document is the sum, over the query terms, of one contribution per term:

$$\text{score} = \sum \ \text{idf} \times \frac{\text{tf} \times (k_1 + 1)}{\text{tf} + k_1 \times \text{norm}}$$

$$\text{norm} = 1 - b + b \times \frac{\text{len}}{\text{avglen}}$$

$$\text{idf} = \ln\left(1 + \frac{N - \text{df} + 0.5}{\text{df} + 0.5}\right)$$

Here `tf` is the weighted count of the term in the document, `len` the document length, `avglen` the average length, `N` the number of documents and `df` the number of documents that contain the term. The constants are $k_1 = 1.5$ and $b = 0.75$. A rare term has a high idf, so it weighs more; repeated occurrences count less and less; and a long document is slightly penalised.

For the query “vibration sensor” and the patent `pat-002` (length 167; N = 10; average length 161.7), the result is built as in Table 10.

Table 10. *Worked BM25 example, taken from the “why this score?” page.*

Term	Count in document	Documents with the term	idf	Contribution
vibration	7	4 of 10	0.894	1.832
sensor	8	5 of 10	0.693	1.454
Score				**3.286**

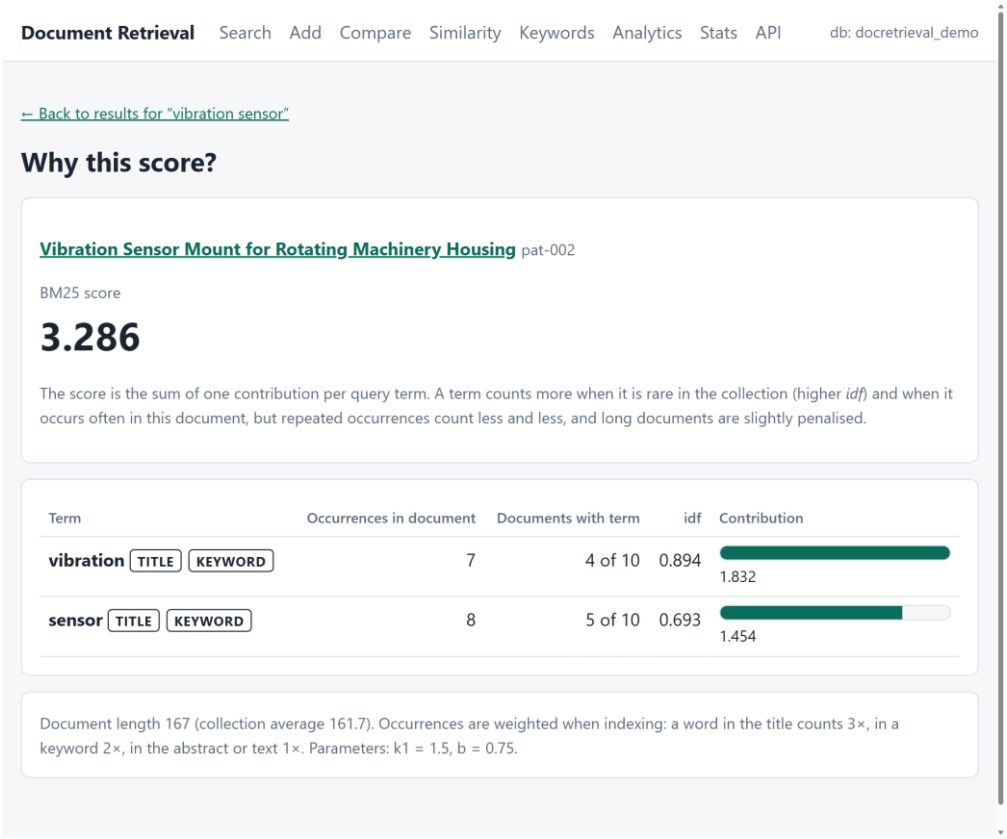


Figure 9. *The "why this score?" page: the score as a sum of one contribution per query term.*

5.4.2 The search procedure

1. The query is parsed into terms, phrases, excluded words and filters.
2. One query on `postings` with `$in` over all query terms fetches the matching entries, using the unique index, and gives each term's document frequency.
3. Each candidate document is scored with BM25.
4. Filters are applied (type, year range, author, keyword, title words), excluded words are removed and phrases are verified in the stored text.
5. The results are ranked, a snippet is cut for each result of the requested page, and facet counts and a spelling suggestion are added.

```
532 by_doc: dict[str, dict[str, int]] = defaultdict(dict)
533 df: Counter = Counter()
534 if pq.terms:
535     for p in self.postings.find({"term": {"$in": pq.terms}}, {"_id": 0}):
536         by_doc[p["doc_id"]][p["term"]] = p["tf"]
537         df[p["term"]] += 1
538     if require_all:
539         by_doc = {d: m for d, m in by_doc.items() if len(m) == len(pq.terms)}
540     if not by_doc:
541         out["suggestion"] = self.suggest(query)
```

Figure 10. *Fetching the postings of all query terms and counting document frequency (`docretrieval.py`, lines 532–541).*

```
scored = []
for doc_id, d in meta.items():
    score, dl = 0.0, d["length"] or 1
    for t, tf in by_doc.get(doc_id, {}).items():
        idf = math.log(1 + (n - df[t] + 0.5) / (df[t] + 0.5))
        score += idf * tf * (K1 + 1) / (tf + K1 * (1 - B + B * dl / avgdl))
    scored.append((score, doc_id))
scored.sort(key=lambda x: (-x[0], x[1]))
```

Figure 11. *The BM25 scoring loop (`docretrieval.py`, lines 553–560).*

5.4.3 Query language, filters, snippets and facets

Table 11. *The query language.*

Write	Meaning
`"posting list"`	the words must appear next to each other
`vibration -sensor`	leave out documents that contain the word *sensor*
`type:patent`	document type (`paper`, `patent`, `report`, `drawing`)
`author:berg`, `author:"tomas berg"`	the author name contains this text
`keyword:"wind turbine"`, `title:gearbox`	has this keyword; the word is in the title
`year:2022`, `year:2020..2023`, `year:>=2021`	publication year, range or comparison

The type, year range and author filters required by the task are available both as query words and as form fields. Each result carries a snippet: the window of 28 words of the text that contains the most query words, with the matches marked. The result page also shows facets (counts by type, year, author and keyword over all matches, not only the visible page); clicking a facet adds the filter to the query. Results are paged, ten to a page, and can be sorted by relevance or by year. If a query word occurs nowhere in the index, the system suggests the indexed word with the smallest edit distance (1 edit for short words, 2 for longer ones) and prefers the one found in more documents.

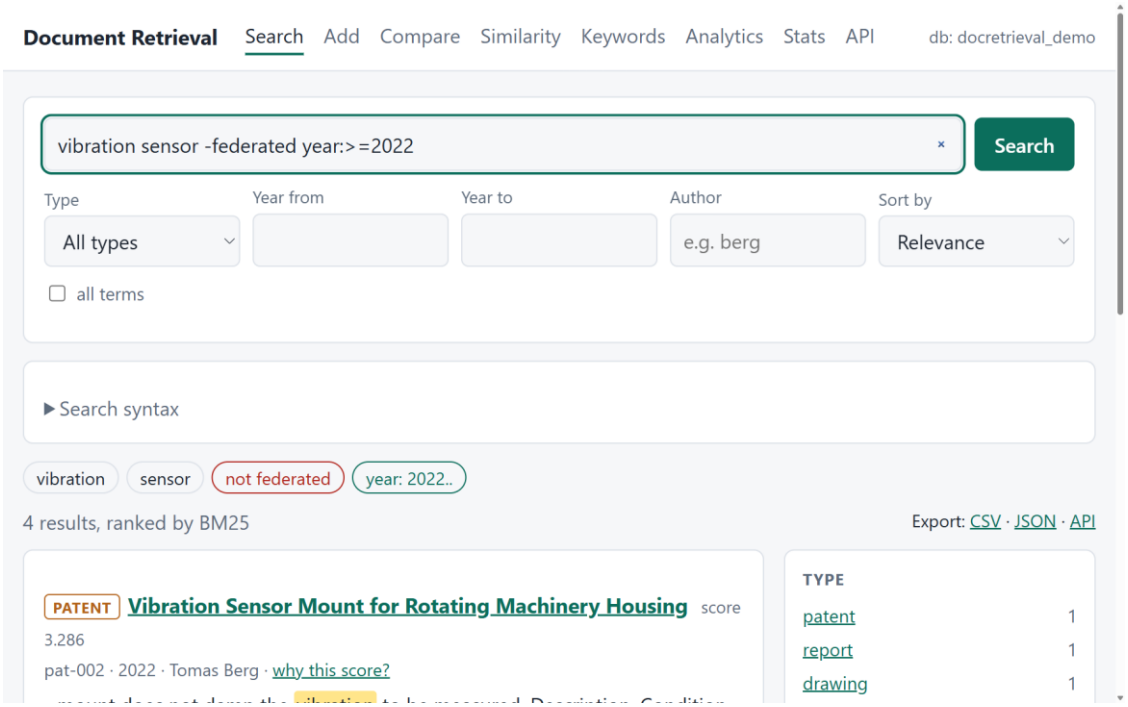


Figure 12. *Search results for `vibration sensor -federated year:>=2022`, with the interpreted query and the facets.*

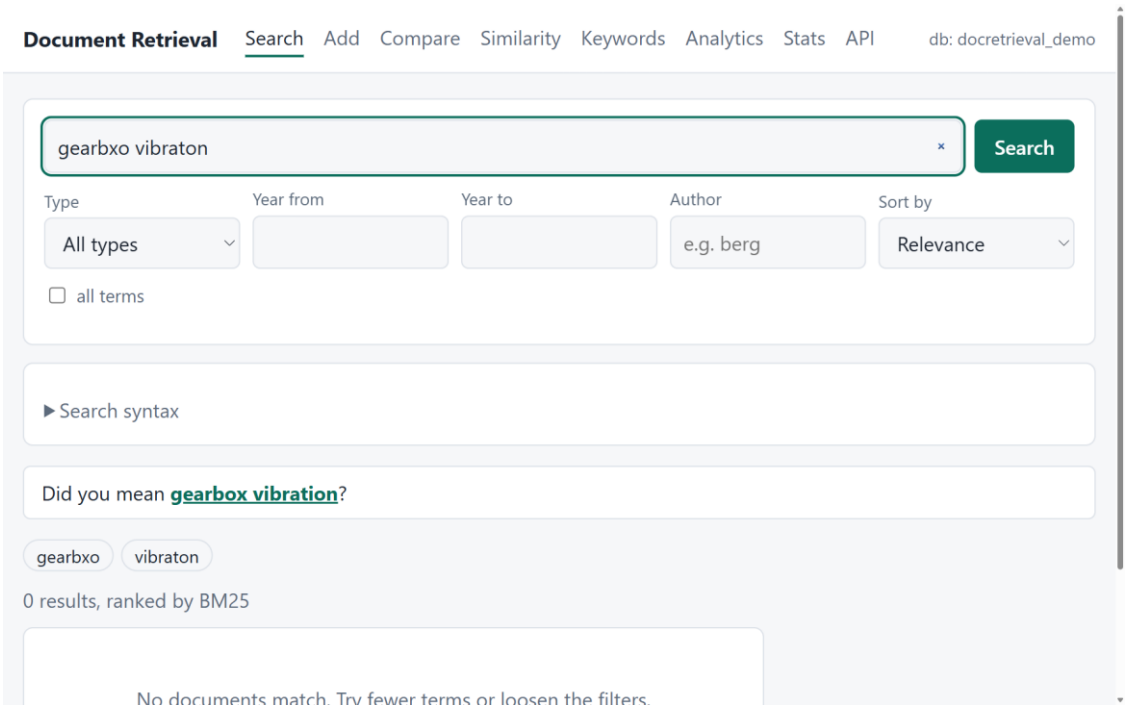


Figure 13. *A misspelled query (`gearbxo vibraton`) with the suggestion "gearbox vibration".*

5.5 Document reading

The stored text is cut into pages of about 1,000 characters, at a paragraph break if there is one near the end of the page and otherwise at a space, so that no word is split. The pages join back into exactly the original text, which is checked by a test. The `read` command and the document page take a page number and report how many pages there are.

5.6 Document comparison

Each document is turned into a vector of term weights: weight = (1 + ln tf) × idf, with a smoothed idf that is never zero. The similarity of two documents is the cosine of the angle between their vectors, that is the dot product divided by the product of their lengths: 0 means nothing in common and 1 means identical. The comparison also lists the terms the documents share (ranked by their joint weight), the terms only one of them has, and the overlap of authors and keywords as a Jaccard value (shared items divided by all items).

```
670 def _tfidf(self, tfs: dict[str, int], n: int, df: dict[str, int]) -> dict[str, float]:
671     return {t: (1 + math.log(tf)) * self._idf_smooth(n, df.get(t, 1)) for t, tf in tfs.items()}
672
673 def compare(self, id_a: str, id_b: str, top_terms: int = 10) -> dict:
674     a, b = self.get_document(id_a, with_text=False), self.get_document(id_b, with_text=False)
675     vecs = self._vectors([id_a, id_b])
676     n, _ = self._corpus()
677     df = self._df(set(vecs[id_a]) | set(vecs[id_b]))
678     wa, wb = self._tfidf(vecs[id_a], n, df), self._tfidf(vecs[id_b], n, df)
679     shared = sorted(set(wa) & set(wb), key=lambda t: -wa[t] * wb[t])
680     only_a = sorted(set(wa) - set(wb), key=lambda t: -wa[t])
681     only_b = sorted(set(wb) - set(wa), key=lambda t: -wb[t])
```

Figure 14. *Building the two weighted vectors and the shared and distinctive terms (`docretrieval.py`, lines 670–681).*

```
200 def _cosine(a: dict, b: dict) -> float:
201     if len(a) > len(b):
202         a, b = b, a
203     dot = sum(w * b[t] for t, w in a.items() if t in b)
204     na = math.sqrt(sum(w * w for w in a.values()))
205     nb = math.sqrt(sum(w * w for w in b.values()))
206     return dot / (na * nb) if na and nb else 0.0
```

Figure 15. *Cosine similarity (`docretrieval.py`, lines 200–206).*

Table 12. *Comparison results for the sample data.*

Documents	Cosine similarity	Comment
`pap-003` (federated fault detection) and `rep-002` (gearbox bearing field report)	0.226	same topic; shared author Tomas Berg; shared keywords: fault detection, wind turbine, vibration
`pap-003` and `pat-001` (posting-list compression patent)	0.031	different topics; nothing shared but filler words

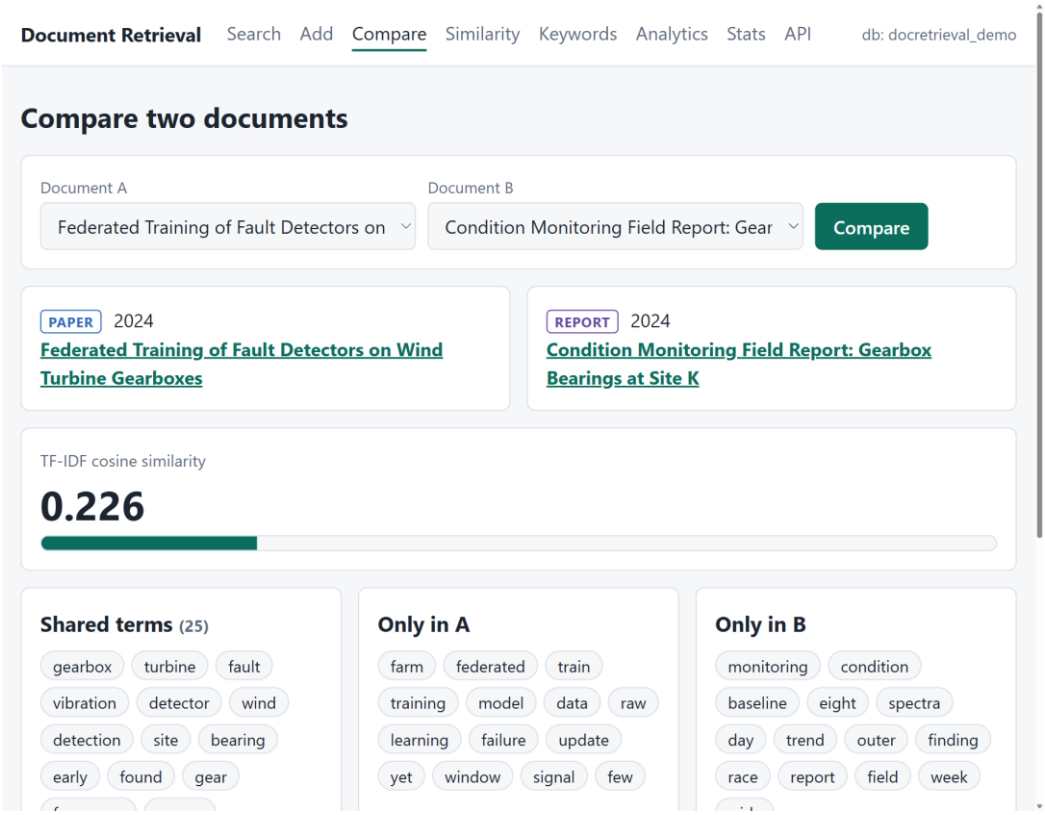


Figure 16. *The comparison page: similarity bar, shared terms, terms only in one document and metadata overlap.*

5.7 Association analysis

Association analysis has three parts. **Related documents** answers “what is like this document?”: the 25 heaviest terms of the document are used to collect candidate documents from the postings, and up to 200 candidates are then scored exactly with the cosine similarity. The **similarity graph** compares every pair of documents once, term by term, from the postings in index order, keeps the pairs whose cosine reaches a threshold as links, and reports the connected groups. **Keyword co-occurrence** counts which keywords appear together in the same document; this is done inside MongoDB with an aggregation pipeline (Figure 17), which projects the keyword list twice, unwinds both copies, keeps each pair once, groups by pair and counts.

```
783 pipeline = [
784     {"$match": match},
785     {"$project": {"a": f"${path}", "b": f"${path}"}},
786     {"$unwind": "$a"}, {"$unwind": "$b"},
787     {"$match": {"$expr": {"$lt": ["$a", "$b"]}}},
788 ]
789 if only:
790     pipeline.append({"$match": {"$or": [{"a": only}, {"b": only}]}})
791 pipeline += [
792     {"$group": {"_id": {"a": "$a", "b": "$b"}, "count": {"$sum": 1}, "docs": {"$push": "$_id"}}},
793     {"$match": {"count": {"$gte": int(min_count)}}},
794     {"$sort": {"count": -1, "_id.a": 1, "_id.b": 1}},
795     {"$limit": int(limit)},
796 ]
```

Figure 17. *The aggregation pipeline that counts keyword pairs (`docretrieval.py`, lines 783–796).*

With a threshold of 0.10 the ten sample documents form 11 links and three groups, which are the three topics of the sample corpus (Table 13). The keyword pairs that occur in most documents are “gearbox + wind turbine” and “vibration + wind turbine” (three documents each).

Table 13. Groups in the similarity graph (cosine ≥ 0.10).

Group	Documents	Topic
1	`drw-001`, `pap-003`, `pat-002`, `rep-002`	wind-turbine gearbox condition monitoring
2	`drw-002`, `pap-001`, `pap-002`, `rep-001`	document databases and replication
3	`pap-004`, `pat-001`	inverted-index retrieval

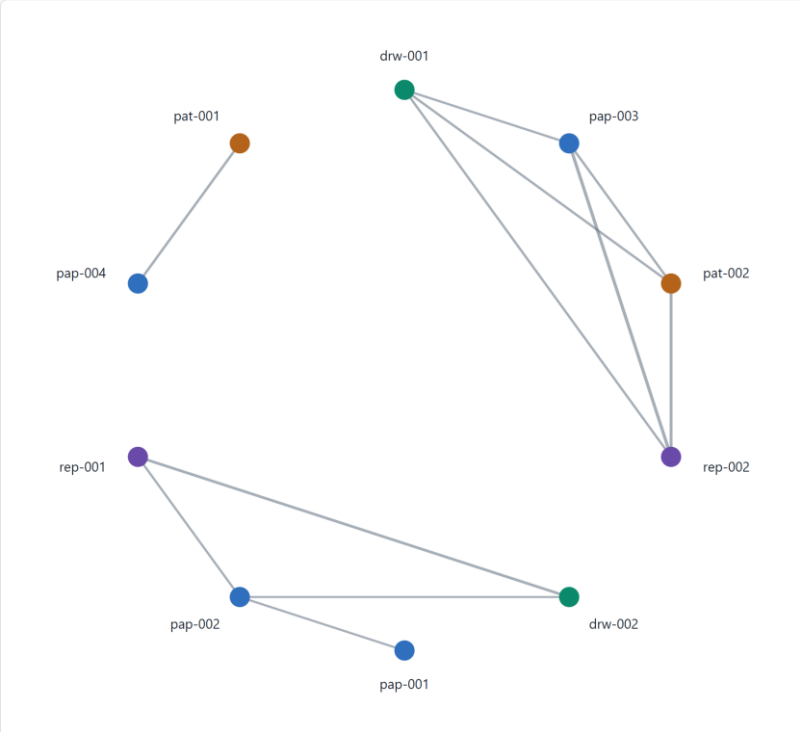


Figure 18. The similarity graph of the sample documents; colours show the document type.

5.8 Document download

The download function writes the original file from GridFS to a folder. The file is copied out in pieces of 64 KB while its SHA-256 is computed; if the result differs from the checksum stored at upload, the partial file is deleted and the download is refused. An existing file is never overwritten (a number is added to the name), and only the base name of the stored file name is used, so that a stored name such as `../../x` cannot write outside the folder.

```
909 h = hashlib.sha256()
910 try:
911     with open(path, "wb") as out:
912         stream = self.fs.open_download_stream(doc["file_id"])
913         for chunk in iter(lambda: stream.read(1 << 16), b""):
914             h.update(chunk)
915             out.write(chunk)
916 except gridfs.errors.NoFile:
917     if os.path.exists(path):
918         os.remove(path)
919     raise DocError(f"GridFS file for {doc_id} is missing")
920 if doc.get("sha256") and h.hexdigest() != doc["sha256"]:
921     os.remove(path)
922     raise DocError(f"checksum mismatch while downloading {doc_id}")
```

Figure 19. Streaming a file out of GridFS with checksum verification (`docretrieval.py`, lines 909–922).

5.9 Deletion

Deletion removes the postings of the document, then its GridFS file, then the document record, and finally subtracts the document from the counters (Section 4.6). Figure 21 shows the real output of the demonstration: a temporary document with a file is added and deleted, and the counts return to 10 documents, 945 postings and 10 stored files, with no posting left for the deleted document.

```
476 doc = self.documents.find_one({"_id": doc_id}, {"text": 0})
477 if not doc:
478     raise NotFound(f"no such document: {doc_id}")
479 removed = self.postings.delete_many({"doc_id": doc_id}).deleted_count
480 file_removed = False
481 if doc.get("file_id") is not None:
482     try:
483         self.fs.delete(doc["file_id"])
484         file_removed = True
485     except gridfs.errors.NoFile:
486         pass
487 self.documents.delete_one({"_id": doc_id})
488 self.meta.update_one({"_id": "corpus"}, {"$inc": {"n": -1, "total_len": -doc["length"]}})
489 self._vocab = None
490 return {"doc_id": doc_id, "postings_removed": removed, "file_removed": file_removed}
```

Figure 20. *delete_document (docretrieval.py, lines 476–490).*

```
$ python docretrieval.py demo      # step 8
=== 8. Delete cleanup: add a throwaway document with a file, then delete it ===
after adding tmp-demo: 11 docs, 951 postings, 11 GridFS files
deleted tmp-demo: 6 postings removed, GridFS file removed
after deleting it : 10 docs, 945 postings, 10 GridFS files
postings still pointing at tmp-demo: 0
```

Figure 21. *Output of the demonstration: add and delete a temporary document.*

5.10 Statistics

The statistics report gives the number of documents by type, the number of postings and distinct terms, the average document length, the number and total size of the GridFS files, the years covered, the number of distinct authors, and the most widespread terms and keywords.

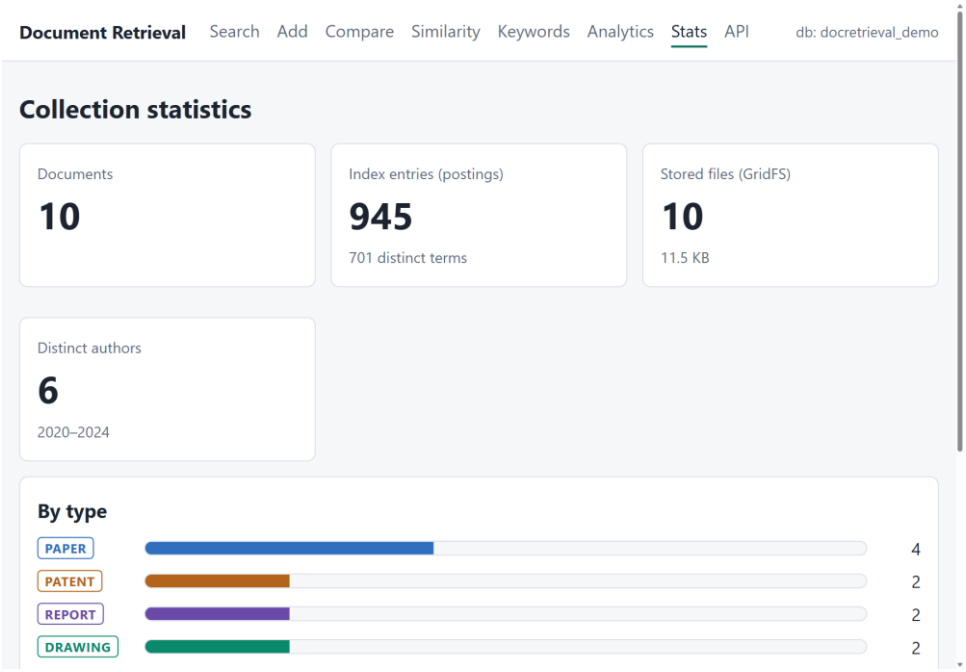


Figure 22. *The statistics page.*

5.11 Command-line interface

The command-line tool has the twelve subcommands required by the task (`add`, `list`, `search`, `read`, `compare`, `related`, `graph`, `cooccur`, `download`, `delete`, `stats`, `demo`) and five more (`explain`, `insights`, `cite`, `duplicates`, `authors`). Appendix A lists them. The connection is configured with `--uri` and `--db` or the environment variables `MONGO_URI` and `DOCRETRIEVAL_DB`.

```
$ python docretrieval.py search "inverted index" compression --limit 2
3 match(es)   [terms: inverted index compression]
 1. [7.436] pat-001  patent, 2020  Method for Compressing Posting Lists Using Block-wise Delta Coding
     authors: Sofia Alvarez
     ...storing a posting list of an **inverted** **index**, the posting list comprising document identifiers in ascen
 2. [4.732] pap-004  paper, 2021  Ranking Short Technical Documents with Sparse and Dense Signals
     authors: Mei Lin, Sofia Alvarez
     ...words are strong evidence, and the **inverted** **index** that supports it is cheap to build and update. Short
... 1 more (use --offset 2)
facets  type: paper=2, patent=1  | year: 2022=1, 2021=1, 2020=1  | author: Sofia Alvarez=2, Mei Lin=2, Omar Haddad=1
```

Figure 23. *Command-line search (real output).*

```
$ python docretrieval.py compare pap-003 rep-002
A: pap-003  Federated Training of Fault Detectors on Wind Turbine Gearboxes (paper, 2024)
B: rep-002  Condition Monitoring Field Report: Gearbox Bearings at Site K (report, 2024)
TF-IDF cosine similarity: 0.2262
Shared terms (25 total, top by joint weight): gearbox, turbine, fault, vibration, detector, wind, detection, site, bea
Only in A: farm, federated, train, training, model, data, raw, learning, failure, update
Only in B: monitoring, condition, baseline, finding, spectra, outer, trend, eight, day, race
Metadata: same type=False, year gap=0
  shared authors (0.33 Jaccard): Tomas Berg
  shared keywords (0.43 Jaccard): fault detection, wind turbine, vibration
```

Figure 24. *Command-line comparison (real output).*

5.12 Web interface and JSON API

The web interface offers every function in the browser: search with filters and facets, the document page with paginated reading, an image preview for drawings and download, a comparison page, the similarity graph, keyword co-occurrence, analytics, statistics, and a form for adding documents. Stored text is HTML-escaped, and only images and PDFs may be displayed inline; any other uploaded file is forced to download, so that an uploaded HTML file cannot run a script in the site. A JSON API

(`/api/search`, `/api/doc/<id>`, `/api/related/<id>`, `/api/compare`, `/api/insights/<id>`, `/api/explain/<id>`, `/api/stats`) gives programs the same results.

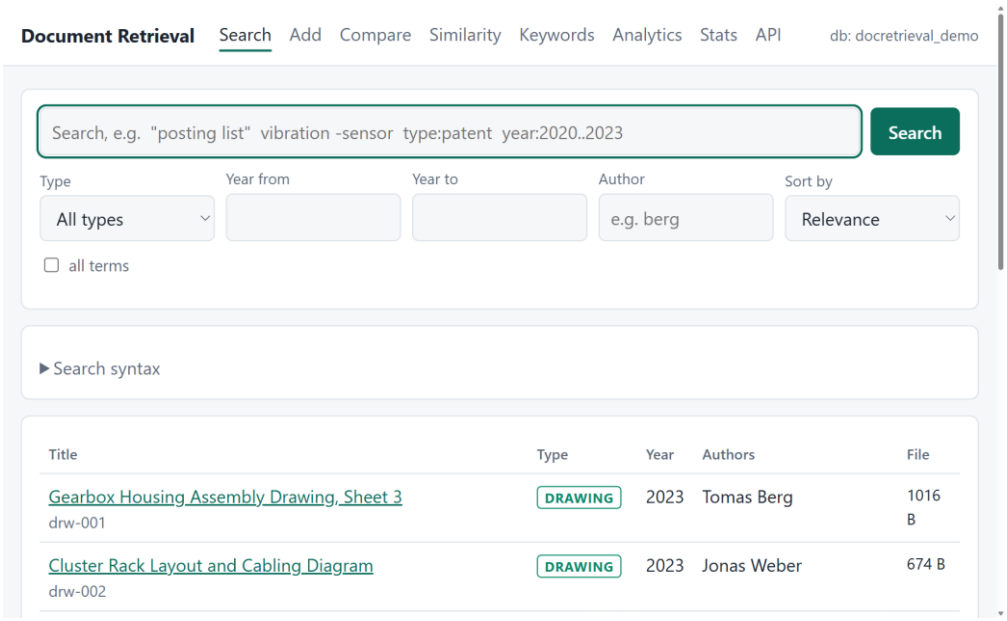


Figure 25. *The document list.*

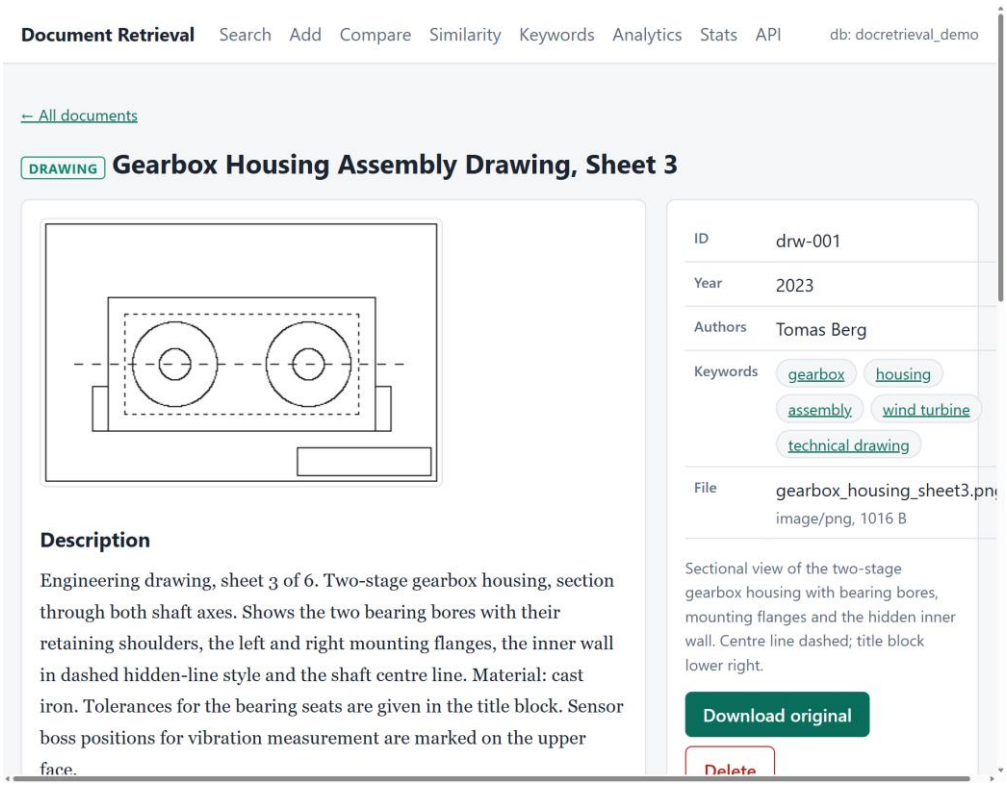


Figure 26. *The page of an engineering drawing: the binary image comes from GridFS.*

5.13 Additional features

- **Document insights:** an automatic summary of three sentences (sentences are scored by the TF-IDF weight of their words and kept in reading order), key terms and the reading time.
- **Analytics:** documents per year by type, the most active authors, and a co-authorship network.

- **Near-duplicate finder:** pairs of documents above a similarity threshold, and a warning when a newly added document is very similar to an existing one.
- **Score explanation:** the page of Figure 9 shows how each result's score is made up.
- **Citations and export:** BibTeX, RIS and APA references, and CSV or JSON export of search results (cells that start with =, +, - or @ are neutralised for spreadsheets).

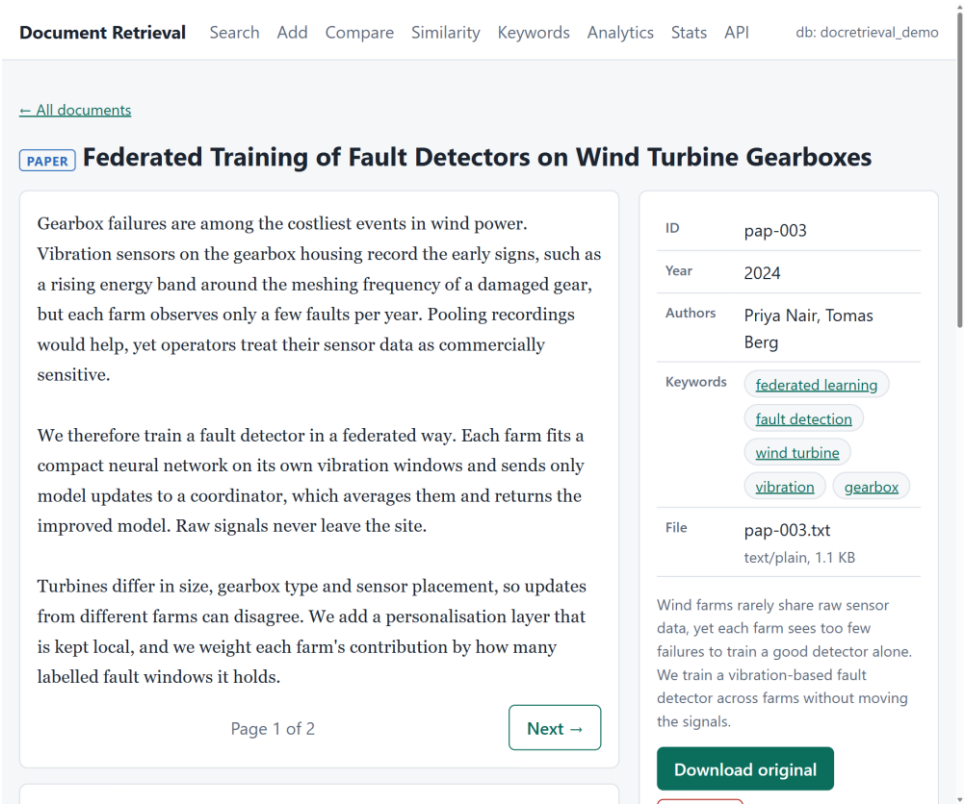


Figure 27. *A paper with the automatic summary, key terms and reading time.*

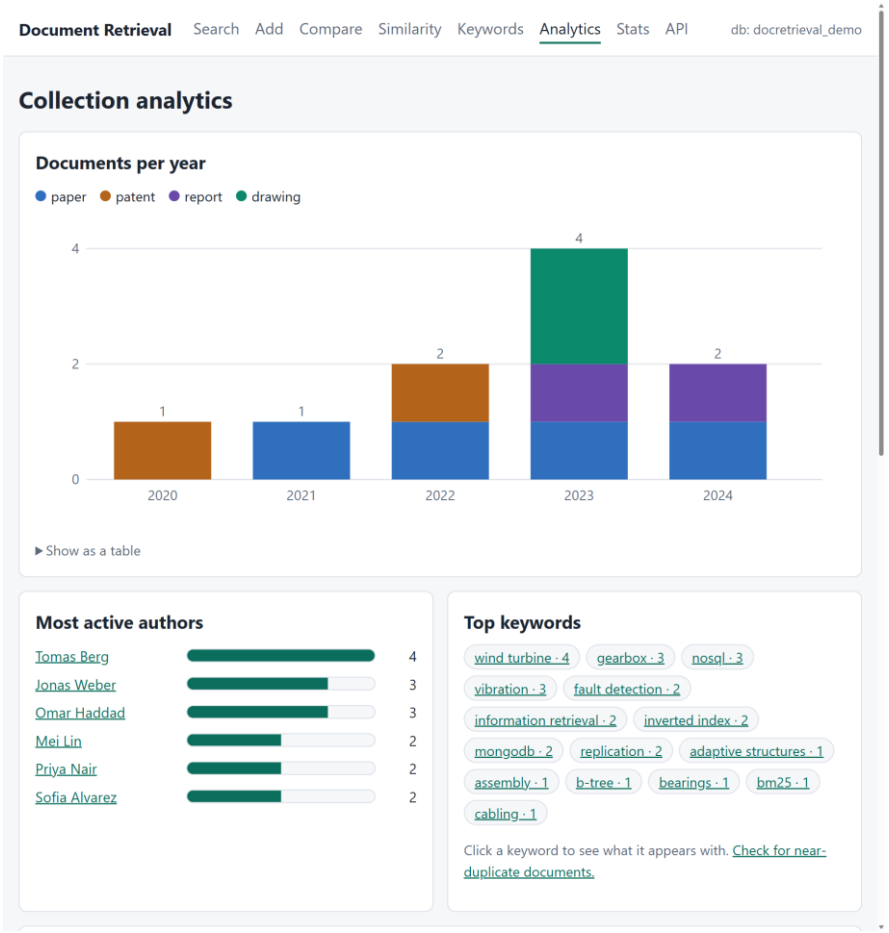


Figure 28. *The analytics page: documents per year by type, top authors, keywords and the co-authorship network.*

5.14 Cloud deployment

`app.py` exposes the Flask application under the name `app`, which Vercel looks for. It reads the connection string from the environment variable `MONGO_URI`. Because a public site has no login, it runs in read-only mode by default: the add and delete pages are removed and any write request is refused with HTTP 403. If the database variable is missing or the database is unreachable, the site shows a “Setup needed” page that says what is wrong.

```
29  uri = os.environ.get("MONGO_URI")
30  if not uri:
31      return _setup_problem("Set the <code>MONGO_URI</code> environment variable to a MongoDB connection string "
32                            "(for example from MongoDB Atlas) and redeploy.")
33  read_only = os.environ.get("READ_ONLY", "1").strip().lower() not in ("0", "false", "no")
34  try:
35      # Indexes are created by whoever loads the data (the CLI does it); a read-only
36      # deployment does not need permission to create them.
37      store = dr.DocumentStore(uri, os.environ.get("DOCRETRIEVAL_DB", dr.DEFAULT_DB), create_indexes=not read_only)
```

Figure 29. *The deployment entry point (`app.py`, lines 29–37).*

6 Testing and evaluation

6.1 Test strategy

All tests run against a real MongoDB server. Each test creates a throw-away database with a random name and drops it afterwards, so tests cannot affect each other or real data. The sample documents

are used as a fixed corpus, so expected results (rankings, similarity values, counts) are known exactly.

Table 14. *The automated tests.*

Test file	Tests	What it covers
`test_docretrieval.py`	54	storage and rollback, unique index, BM25 against a hand computation, filters, reading, comparison, association analysis, download round trip, delete cleanup, statistics
`test_features.py`	28	query language, phrases, exclusion, facets, paging, spelling suggestions, score explanation, insights, analytics, citations, duplicates
`test_webapp.py`	24	every web page, search and filters in the browser, file download, escaping, upload then search then delete, read-only mode
`test_webapp_features.py`	21	facet links, pagination, exports, citation downloads, analytics and duplicate pages, JSON API and its errors
Total	**127**	

```
$ python -m pytest -q <file>        # real results, one run per test file
test_docretrieval.py      54 passed in 9.70s
test_webapp.py            24 passed in 8.14s
test_features.py          28 passed in 4.52s
test_webapp_features.py   21 passed in 6.25s
total                     127 passed
```

Figure 30. *Result of the test run (real output).*

6.2 Checking that the tests can fail

To make sure the tests detect real faults, the code was broken on purpose and the tests were run. Each change was detected (Table 15), and the original code passes again.

Table 15. *Deliberate faults and the tests that caught them.*

Change made to the code	Result
do not remove the postings and the GridFS file when deleting a document	5 delete tests failed
remove the length normalisation from the BM25 formula	the hand-computed BM25 test failed
accept documents without checking the exact phrase	the phrase-search test failed
do not remove documents that contain an excluded word	the exclusion test failed
allow a larger edit distance for spelling suggestions	the spelling-suggestion test failed

6.3 Demonstration on the sample data

Table 16. *Results for the ten sample documents.*

Action	Result
search `inverted index compression`	`pat-001` 7.436, `pap-004` 4.732, `pap-001` 2.420
search `"posting list"`	only `pat-001`
search `vibration -federated`	3 results: `pat-002`, `rep-002`, `drw-001` (`pap-003` is excluded)
search `gearbxo vibraton`	no result; suggestion “gearbox vibration”
compare `pap-003` and `rep-002`	cosine 0.226 (unrelated pair: 0.031)
similarity graph, threshold 0.10	11 links, 3 topic groups
download `drw-001`	1,016 bytes written; SHA-256 verified
delete a temporary document	6 postings and its file removed; counts return to 10 / 945 / 10

6.4 Performance at scale

6.4.1 Method

A synthetic corpus of 2,000, 5,000 and 10,000 documents was loaded into an empty database on one computer (MongoDB 8.0.34 standalone, Windows 11, Python 3.12.10). Each document has 110 to 190 words drawn from the vocabulary of the sample documents with a skewed (Zipf-like) frequency, a title, one to three authors from a pool of 300, three to five keywords from a pool of 120, one of the four types and a year from 2010 to 2025; the text is also stored as a file in GridFS. Queries were chosen by the real document frequency of the terms: “frequent” terms are the ten most widespread index terms (found in 8,944 to 10,000 of 10,000 documents), “medium” terms are around the median (about 800 documents), and “rare” terms are the least widespread (400 to 570 documents). Each query type was run 100 times after a warm-up; related documents were measured on 30 documents. The tables give medians.

6.4.2 Size and ingest

Table 17. *Collection size and ingest speed.*

Documents	Index entries	Data size	Index size	Time to load	Documents per second	Median add
2,000	197,824	19.4 MB	8 MB	9.9 s	202.5	4.6 ms
5,000	493,474	48.3 MB	17.4 MB	25.9 s	193.2	4.77 ms
10,000	986,895	96.5 MB	38.6 MB	51.6 s	193.6	4.79 ms

Ingest speed is constant at about 194 documents per second (a median of 4.79 ms to store one document with its file and about 100 index entries), so loading time grows linearly with the number of documents. About 99 index entries are created per document.

6.4.3 Search and other functions

Table 18. Median (95th percentile) time in ms of searches, ten results per search with snippets.

Search	2,000 documents	5,000 documents	10,000 documents
search, 2 frequent terms	34.94 (40.79)	75 (96.73)	180.67 (219.82)
search, 2 medium terms	9.55 (11.03)	16.26 (25.01)	27.34 (40.42)
search, 2 rare terms	8.1 (11.86)	8.85 (10.6)	18.79 (23.31)
search, 1 frequent + 2 rare terms	28.62 (31.81)	64.13 (80.51)	151.81 (195.67)
search with type and year filters	14.99 (16.26)	28.28 (38.13)	63.58 (79.24)
search with author filter	20.08 (22.34)	40.15 (54.71)	88.49 (118.24)
phrase search (2 medium terms)	19.95 (23.22)	45.14 (60.01)	88.24 (117.82)

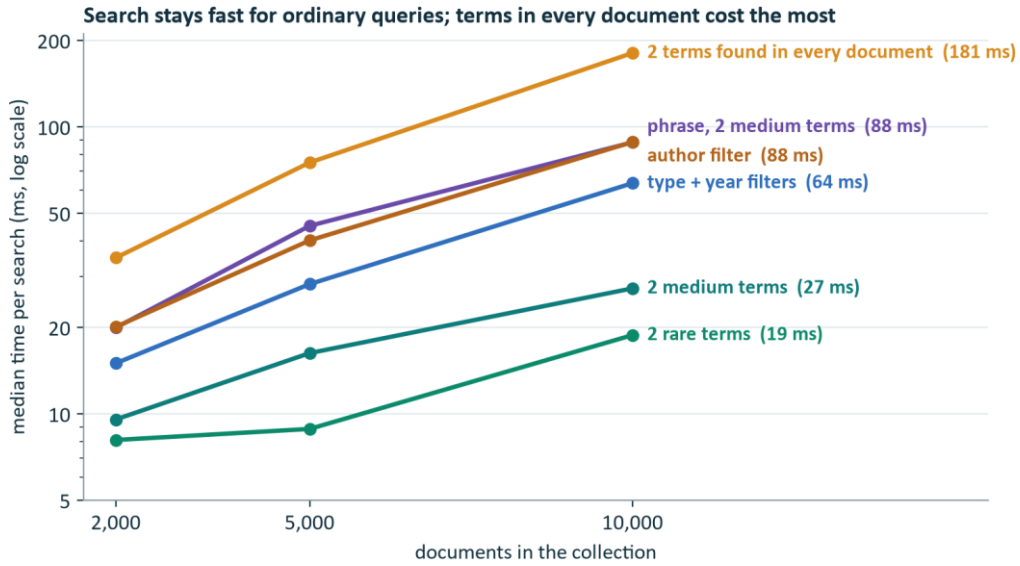


Figure 31. Median search time against collection size.

Table 19. Median time in ms of the other functions.

Function	2,000 documents	5,000 documents	10,000 documents
related documents	168.41	337.54	665.55
compare two documents	41.67	86.03	184.34
read one page	0.31	0.26	0.36
download (GridFS + SHA-256)	2.4	2.52	2.52
keyword co-occurrence (whole collection)	46.02	116.44	217.11
statistics	129.23	323.88	639.27
delete (document, postings and file)	4.45	2.94	4.12

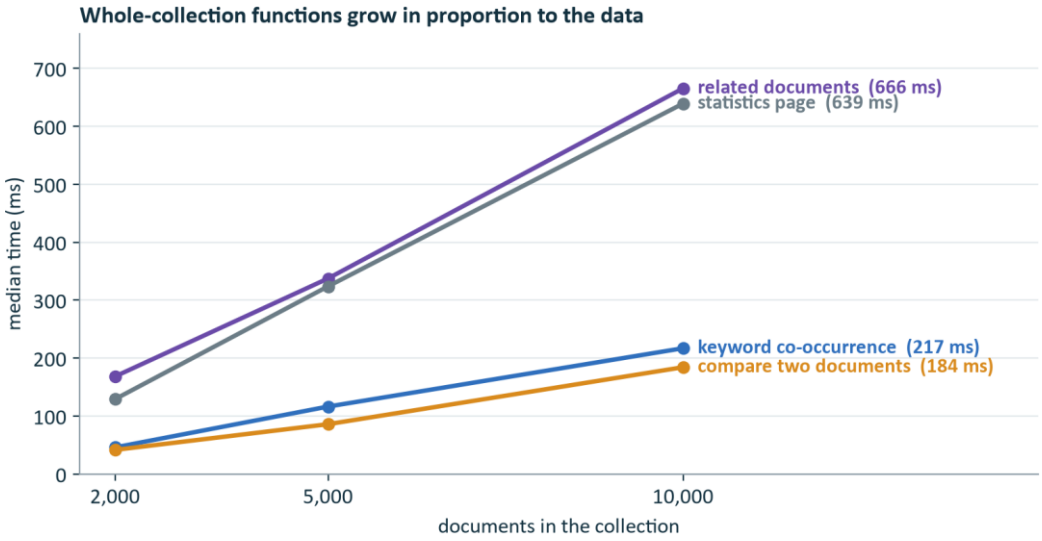


Figure 32. *Functions that read many index entries or documents grow in proportion to the data.*

6.4.4 What the index buys

Figure 33 compares fetching the postings of one term through the (term, doc_id) index with forcing a scan of the whole collection, which holds 986,895 postings. For a term found in every document the lookup takes 29 ms against 422 ms for the scan; for a rarer term (507 postings) it takes 4 ms against 479 ms. Both give identical results, and MongoDB's query plan for the lookup uses an index scan.

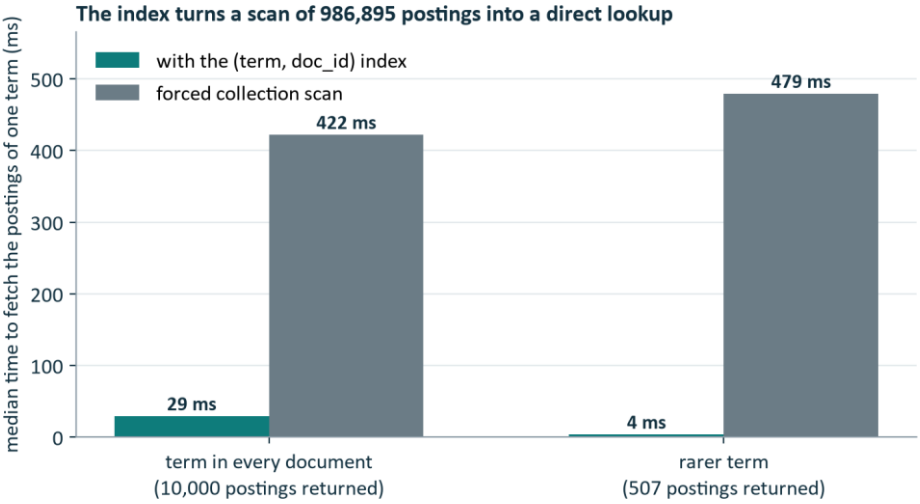


Figure 33. *Fetching the postings of one term: index lookup against a collection scan (10,000 documents).*

6.4.5 Observations

- **Ingest is linear**, about 194 documents per second at every size.
- **Search cost follows the number of matching index entries.** With 10,000 documents, a search for two rare terms takes 18.79 ms and for two medium terms 27.34 ms. A search for two terms that occur in every document must score all 10,000 documents and takes 180.67 ms. The synthetic vocabulary is small (657 terms), so its frequent terms are far more widespread than in real text; real collections have a longer tail of rarer terms.

- **Reading and download do not depend on the collection size:** a page is read in 0.36 ms and a file is downloaded and verified in 2.52 ms.
- **Deletion is cheap and clean:** 4.12 ms per document; after deleting 100 documents 9,814 postings and 100 files were gone, and no posting was left for a deleted document.
- **Four functions grow roughly in proportion to the data** (related documents 665.55 ms, comparison 184.34 ms, whole-collection keyword counting 217.11 ms and statistics 639.27 ms at 10,000 documents). Comparison and related documents are slow mainly because the document frequency of every term is counted from the postings each time.

Limits of the measurement. All values come from one computer with one MongoDB server and a synthetic corpus, so they show how the system scales and where its costs lie, not absolute performance in production.

7 Discussion

7.1 What worked well

- Storing the files, the records and the index in one database made all-or-nothing storage and clean deletion simple, and a single backup covers everything.
- A custom inverted index in a unique-indexed collection gave full control over ranking and made it possible to explain every score.
- Keeping all logic in one engine class let three interfaces and 127 tests share the same code.

7.2 Limitations

- Only plural endings are removed from words; “optimize” and “optimization” are different terms.
- All candidate documents of a query are scored. A term that occurs in every document makes a search proportional to the collection size (181 ms at 10,000 documents).
- Document frequencies are counted from the postings at query time, which makes comparison and related-document search slower as the index grows.
- The similarity graph compares all pairs and is meant for thousands, not millions, of documents.
- A single MongoDB server has no multi-document transactions, so failed writes are undone by hand.
- Text is extracted only from `.txt` and `.pdf` files; scanned documents and drawings are found through their title, keywords and description.
- The web site has no login; a public deployment is therefore read-only.

7.3 Future work

- **Store the document frequency** of every term in a vocabulary collection, updated when documents are added and deleted, so that comparison and related-document search no longer count postings.

- **Limit scoring** to the best candidates (top-k pruning) and treat the most widespread terms like stop words.
- A full stemmer and a synonym list; Atlas Search or vector search to match by meaning.
- A login with an administrator role so that writes can be allowed safely; transactions on a replica set; OCR for scanned documents and drawings.

8 Conclusion

The experiment produced a document retrieval system on MongoDB that stores four kinds of documents and provides storage, retrieval, reading, comparison, association analysis and download, together with deletion that cleans up the index and the stored files. The design uses MongoDB features for what they are good at: embedded lists for authors and keywords, a unique compound index for the inverted index, GridFS for binary files and an aggregation pipeline for keyword pairs.

The system is tested by 127 automated tests against a real database, and measurements with up to 10,000 documents show linear ingest, short search times for ordinary queries, and the large benefit of the index. The measurements also located the main weaknesses (scoring all candidates of very common terms, and counting document frequencies at query time) and the next steps that would remove them.

References

1. S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond”, Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.
2. C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.
3. G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval”, Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.
4. MongoDB Inc., MongoDB Manual: GridFS, aggregation pipeline and indexes, https://www.mongodb.com/docs/manual/.
5. PyMongo documentation, https://pymongo.readthedocs.io/.
6. Flask documentation, https://flask.palletsprojects.com/.

Appendix A: Command reference

Table 20. *Subcommands of* `python docretrieval.py`.

Command	Purpose	Example
`add`	store a document with its file	`add --title T --type paper --file a.pdf`
`list`	list the stored documents	`list --type patent`
`search`	BM25 search with filters and snippets	`search vibration --type report --year-from 2023`
`read`	read a document page by page	`read pap-003 --page 2`

Command	Purpose	Example
`compare`	compare two documents	`compare pap-003 rep-002`
`related`	documents most similar to one	`related pat-002`
`graph`	similarity graph above a threshold	`graph --threshold 0.10`
`cooccur`	keyword co-occurrence counts	`cooccur --min-count 2`
`download`	write the original file to a folder	`download drw-001 --dest downloads`
`delete`	delete documents with index entries and files	`delete pap-001`
`stats`	collection, index and storage statistics	`stats`
`demo`	load the sample data and show every function	`demo`
`explain`	break a document's BM25 score into its terms	`explain pat-002 vibration sensor`
`insights`	key terms, summary and reading time	`insights pap-003`
`cite`	BibTeX, RIS or APA reference	`cite pap-003 --format ris`
`duplicates`	near-duplicate pairs	`duplicates --threshold 0.6`
`authors`	documents per year, top authors, co-authorship	`authors`

Appendix B: Sample documents

Table 21. *The ten sample documents (all text is original).*

Id	Type	Year	Title	Authors
`pap-001`	paper	2022	Adaptive Learned Indexes for Skewed Key Distributions	Mei Lin, Omar Haddad
`pap-002`	paper	2023	Sharding Strategies for Document Stores under Hotspot Workloads	Omar Haddad, Priya Nair
`pap-003`	paper	2024	Federated Training of Fault Detectors on Wind Turbine Gearboxes	Priya Nair, Tomas Berg
`pap-004`	paper	2021	Ranking Short Technical Documents with Sparse and Dense Signals	Mei Lin, Sofia Alvarez
`pat-001`	patent	2020	Method for Compressing Posting Lists Using Block-wise Delta Coding	Sofia Alvarez
`pat-002`	patent	2022	Vibration Sensor Mount for Rotating Machinery Housing	Tomas Berg
`rep-001`	report	2023	Load Test Report: Replica Set Failover in a Three-Node Document Cluster	Omar Haddad, Jonas Weber
`rep-002`	report	2024	Condition Monitoring Field Report: Gearbox Bearings at Site K	Tomas Berg, Jonas Weber
`drw-001`	drawing	2023	Gearbox Housing Assembly Drawing, Sheet 3	Tomas Berg
`drw-002`	drawing	2023	Cluster Rack Layout and Cabling Diagram	Jonas Weber

Appendix C: Benchmark tables

Table 22. *Number of documents that contain the query terms used at each size.*

Documents	Documents that contain the term
2,000	frequent terms: 1,787 to 2,000; medium terms: 163 to 171; rare terms: 75 to 115
5,000	frequent terms: 4,455 to 5,000; medium terms: 407 to 427; rare terms: 200 to 288
10,000	frequent terms: 8,944 to 10,000; medium terms: 814 to 856; rare terms: 397 to 570

Table 23. *Index lookups: with the index and with a forced collection scan (median of 20 runs).*

Documents	Term	Postings returned	With index (ms)	Collection scan (ms)
2,000	in every document	2,000	6.97	75.97
2,000	rarer term	100	1.53	99.51
5,000	in every document	5,000	13.09	191.94
5,000	rarer term	255	2.33	178.71
10,000	in every document	10,000	29.25	422.05
10,000	rarer term	507	3.63	479.37

The script that produced the measurements is `docs/benchmark.py`; the raw results are in `docs/benchmark_results_*.json`.

Appendix D: Running the system

1. Install and start MongoDB (for example `docker run -d -p 27017:27017 mongo`, or a local installation), or use a MongoDB Atlas connection string.
2. Install the packages: `pip install -r requirements.txt`.
3. Load the sample data and see every function: `python docretrieval.py demo`.
4. Start the web site: `python webapp.py --db docretrieval_demo` and open http://127.0.0.1:5000.
5. Run the tests (they need MongoDB): `pip install -r requirements-dev.txt`, then `python -m pytest -q`.